

LSTM-Based Smart Energy Consumption Forecasting Model

Documentation prepared by: Kavana Reddy M

1. INTRODUCTION

This document provides a detailed explanation of the design, implementation, training, evaluation and outcomes of an LSTM-based time series forecasting model developed for smart energy consumption analysis. The project is part of Module 5 of the Infosys Springboard AI Internship and focuses on understanding not just how the model works, but why each step is necessary.

The objective of this project is to predict future energy consumption values based on historical hourly energy usage data using a Long Short-Term Memory (LSTM) neural network. LSTM is chosen due to its ability to capture long-term dependencies in sequential and time-series data, which is critical for accurate energy forecasting.

2. PROBLEM STATEMENT

Accurate energy consumption forecasting is essential for smart energy management systems. Traditional machine learning models often fail to capture temporal dependencies present in time-series data. This project aims to overcome that limitation by implementing an LSTM-based deep learning model that can learn patterns from historical energy consumption data and generate reliable future predictions.

3. DATASET DESCRIPTION

The dataset consists of historical hourly energy consumption values. Each data point represents the amount of energy consumed during a specific hour.

Key characteristics of the dataset:

- Type: Time-series data
- Frequency: Hourly
- Target variable: Energy consumption (in kWh)
- Nature: Univariate (only one feature is used)

Before training, the data is pre-processed through scaling and sequence generation to make it suitable for LSTM input.

4. DATA PREPROCESSING

4.1 Data Scaling

Energy consumption values are scaled using Min-Max scaling. This step is essential because neural networks perform better when input values are within a small, consistent range (typically between 0 and 1). Scaling improves training stability and convergence speed.

4.2 Sequence Creation (Sliding Window Approach)

LSTM models require sequential input. Therefore, the time-series data is converted into overlapping sequences using a sliding window technique.

- Time steps (look-back window): 24
- Each input sequence contains energy consumption values from the previous 24 hours
- The model predicts the energy consumption value for the next hour

This transformation converts the dataset into supervised learning format.

5. LSTM INPUT SHAPE EXPLANATION

The LSTM model expects input data in a 3-dimensional format:

(samples, timesteps, features)

Explanation of each dimension:

1. Samples - Represent the total number of independent input sequences generated from the dataset.
 - Training samples: 79,980 sequences
 - Testing samples: 19,996 sequences

Each sample corresponds to one 24-hour historical window.

2. Timesteps - Represent the number of past observations the model looks at for each prediction.
 - Timesteps = 24

This means the model learns patterns across 24 consecutive hourly energy values.
3. Features - Represent the number of variables at each timestep.
 - Features = 1

Only energy consumption is used. No additional variables (such as temperature or day of the week) are included.

6. MODEL ARCHITECTURE

The forecasting model is built using an LSTM neural network with the following structure:

- Input Layer: Accepts sequences of shape (24, 1)
- LSTM Layer: Learns temporal dependencies and patterns
- Dense Output Layer: Produces a single continuous value representing predicted energy consumption

The LSTM architecture is well-suited for time-series problems due to its internal memory cells and gating mechanisms that prevent vanishing gradient issues.

7. MODEL TRAINING CONFIGURATION

7.1 Learning Rate

The learning rate controls how much the model's weights are updated during training.

- A smaller learning rate ensures stable learning but slows convergence
- A larger learning rate speeds up learning but risks overshooting optimal values

A balanced learning rate is used to ensure efficient and stable training.

7.2 Epochs

An epoch represents one complete pass through the entire training dataset. The model is trained for multiple epochs to allow it to gradually learn patterns in the data.

7.3 Batch Size

Batch size determines how many samples are processed before updating the model weights.

- Smaller batch sizes provide more frequent updates but increase training time
- Larger batch sizes improve efficiency but may reduce generalization

An optimal batch size is selected to balance performance and efficiency.

8. EARLY STOPPING REGULARIZATION

EarlyStopping is used to prevent overfitting and unnecessary training.

- Monitored metric: Validation loss
- Training stops when validation loss stops improving for a predefined number of epochs
- The model restores the best-performing weights automatically

This ensures that the final model generalizes well to unseen data.

9. MODEL EVALUATION

The trained model is evaluated on the test dataset after inverse-transforming the scaled predictions.

Evaluation metrics used:

1. Mean Absolute Error (MAE)
 - Achieved MAE: 0.4496 kWh
 - Represents the average absolute difference between predicted and actual energy values
 - Easy to interpret and robust to outliers
2. Root Mean Squared Error (RMSE)
 - Achieved RMSE: 0.5196 kWh
 - Penalizes larger errors more heavily
 - Useful when large prediction errors are particularly undesirable

Both metrics indicate that the LSTM model provides accurate and reliable predictions.

10. TRAINING VS VALIDATION LOSS ANALYSIS

A plot of training loss and validation loss was generated to analyze model learning behavior.

Observations:

- Training loss steadily decreases, indicating effective learning
- Validation loss closely follows training loss
- No significant divergence between losses, suggesting minimal overfitting

This confirms that the model generalizes well.

11. MODEL SAVING AND REUSABILITY

The trained model is saved as:

lstm_energy_model.h5

Benefits of saving the model:

- Avoids retraining from scratch
- Enables reuse for inference and deployment
- Supports integration into real-world applications

A warning suggests using the newer `.keras` format, which is recommended for future projects.

12. PRACTICAL APPLICATIONS

This LSTM-based forecasting model can be used in:

- Smart grid energy management systems
- Demand prediction for power utilities
- Energy optimization in smart homes
- Load balancing and peak demand analysis

By forecasting energy usage accurately, organizations can reduce waste, lower costs, and improve sustainability.

13. INSIGHTS AND FUTURE ENHANCEMENTS

- Compare LSTM performance with baseline models such as Linear Regression to quantify performance improvement
- Incorporate additional features such as temperature, weekday/weekend, or holidays
- Deploy the model using a Flask or FastAPI application for real-time forecasting
- Convert the model to the `.keras` format for long-term compatibility

14. CONCLUSION

This project successfully demonstrates the application of LSTM networks for time-series energy forecasting. Through careful preprocessing, thoughtful model design, and proper evaluation, the model achieves strong predictive performance. The project highlights the importance of understanding both theoretical concepts and practical implementation, making it suitable for real-world smart energy applications.